

Practical Work No. 14

DS Stack

1. Définition

A stack (Pile en Fr) is defined as a set of data, of variable size, possibly empty, on which the following operations can be performed:

- **create_stack**: creates an empty stack.
- **is_empty**: checks whether a stack is empty or not.
- **push (empiler en Fr)** : adds an element of type T to the stack.
- **pop (depiler en Fr)**: removes an element from the stack.
- **last (dernier en Fr)** : returns the last pushed element that has not yet been popped.

In general, the operations applicable to a data structure are divided into three categories:

- **Creation operations**: allow the creation of a data structure (DS).
 - Example: DS stack → create stack.
- **Consultation operations**: provide information about a DS.
 - Example: DS stack → is_empty and last. A consultation operation does not modify the DS.
- **Modification operations**: act on the DS.
 - Example: push and pop.

ILLEGAL OPERATIONS:

Some operations defined on a data structure require **preconditions**: these operations cannot be applied systematically to a data structure.

Illustration: * The **pop** operation requires the stack to be non-empty.
* The same applies to the **last** operation.

2. Properties

The operations defined on the stack data structure follow these properties:

- **(P1)** `create_stack` creates an empty stack.
- **(P2)** If an element is added (using `push`) to a stack, the resulting stack is non-empty.
- **(P3)** A `push` operation immediately followed by a `pop` leaves the stack unchanged.
- **(P4)** Elements are retrieved from a stack in the reverse order of their insertion (`push`).
- **(P5)** A `pop` operation immediately followed by `push` (with the popped element) leaves the stack unchanged.

Note: These properties define the semantics (behavior or role) of the operations applicable to the stack data structure.

Conclusion: The stack data structure follows the **LIFO** (Last In, First Out) principle, also known as **DRPS** (Dernier Rentré, Premier Sorti).

3. Physical Representation of a Data Structure

To materialize (concretize, implement) a data structure, two types of representation are distinguished:

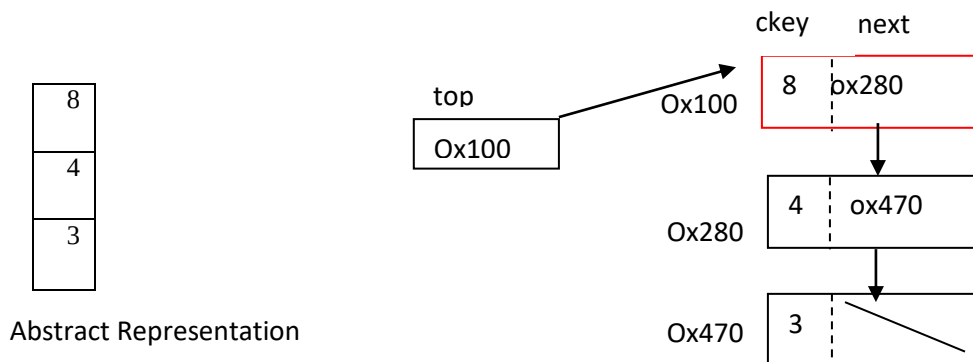
- **Linked Representation:** The elements of a data structure are placed at arbitrary locations (in main memory) but are linked together. → pointers.
- **Contiguous Representation:** The elements of a data structure are stored in a contiguous space. → array.

3.1 Linked Representation of the Stack Data Structure

Pointers are used to explicitly link the elements forming the data structure.

The linked representation consists of multiple nodes. Each node contains the following fields:

- The "**key**" (**cle in Fr**) field stores an element of the stack.
- The "**next**" (**next in Fr**) field points to the previously pushed element.
- The "**top**" (**sommet in Fr**) points to (or identifies) the element at the top of the stack.



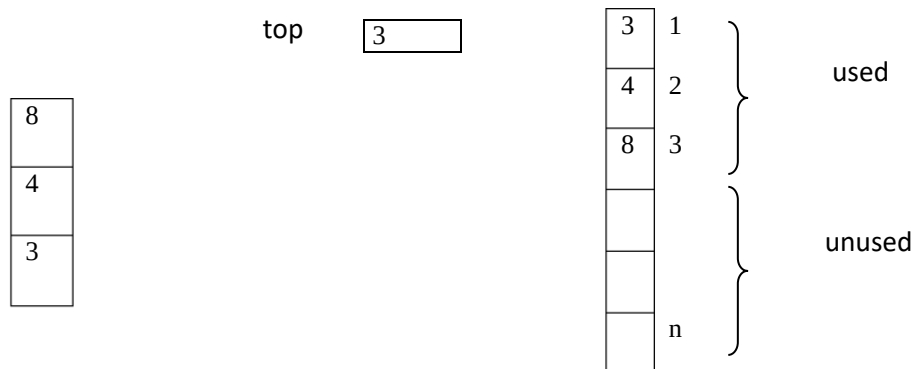
3.2 Contiguous Representation

To represent the stack data structure contiguously, the concept of an array is used.

Since the number of elements in an array must be fixed before use, there are two parts:

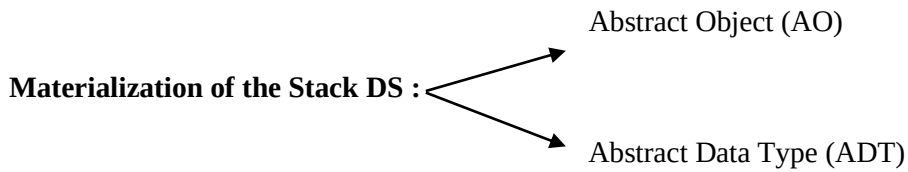
- **Used part:** It spans from index 1 to **top**.
- **Unused part:** It spans from **top + 1** to **n**.

Where **top** is the index, which ranges from 0 to **n**.



Représentation abstraite

4. Materialization of the Stack Data Structure



- Abstract Object (AO) → Single instance (here, a single stack) → unique → implicit.
- Abstract Data Type (ADT) → Multiple instances → it is necessary to mention explicit the current instance.

4.1 Data Structure Stack as an Abstract Object

The stack interface groups services exported by this interface. Each service corresponds to an operation that can be applied to the data structure (here, the Stack data structure).

<pre> /* pile.h is supposed to be stored in the current directory called the working directory */ #include<stdlib.h> #include<assert.h> /* physical representation */ struct element { int key ; struct element * next ; } ; static struct element *top ; /* To restrict the scope of this variable to this file, the reserved word static must be used. It means that the variable 'top' is not intended for export */ void create_stack (void) { top=NULL ; } unsigned is_empty (void){ return(top==NULL) ; } int last(void) { assert(! is_empty()) ; return(top ->key) ; } void push(int info) { struct element*p ; p=(struct element*)malloc(sizeof(struct element)) ; p->key=info; p->next=top; /* update top */ top=p; } void pop (void) { struct element* p ; assert (!is_empty()); p=top ; top= top ->next ; free(p) ; } </pre>	<pre> *****Usage***** #include<stdio.h> #include<assert.h> #include " stack.h " /* we are using services offered by the stack interface */ void main(void) { unsigned i; create_stack(); assert(is_empty ()); for(i=1;i<=10;i++) { push (i); } assert(!is_empty ()); for(i=1;i<=10;i++) { printf(" %d \n" , last ()); pop (); } assert (is_empty ()); } </pre>
--	---

4.2 Stack data structure as an abstract data type

In this paragraph, we will concretize the stack data structure as an abstract data type capable of managing multiple instances of the stack data structure, rather than just a single instance (abstract object, see 4.1)

<pre> ***** Interface Part: stack.h *****/ /* physical representation */ struct element { int key ; struct element* next ; } ; /* operations or exported services */ struct element* create_stack (void) ; unsigned is_empty (struct element*) ; int last (struct element*) ; void push (int, struct element**) ; void pop (struct element**) ; ***** Implementation: Stack.c *****/ #include <stdlib.h> #include <assert.h> #include "stack.h " /* realization of the operations exported by stack.h */ struct element* create_stack (void) { return NULL; } unsigned is_empty (struct element* top) { return(top==NULL); } int last (struct element* top) { assert(!is_empty(top)) ; return(top->key) ; } void push (int info, struct element** top) { struct element* q ; q=(struct element*) malloc(sizeof(struct element)) ; q->key=info; q->next=*top; /* update */ *top=q; } void pop(struct element **top) { struct element *q; assert(!is_empty(*top)); q=*top ; *top=(*top) -> next ; free(q) ; } </pre>	<pre> ***** Usage Part: test.c *****/ #include<stdio.h> #include "stack.h" void main (void) { struct element* p1; struct element* p2; unsigned i; p1=create_stack(); for (i=1;i<=10;i++) { push(i,&p1); } p2=create_stack(); for (i=11;i<=20;i++) { push(i,&p2); } /* display p1 and p2 */ for (i=1 ;i<=10 ;i++) { printf(" %d\t%d\n" ,last(p1),last(p2)); pop(&p1); pop(&p2); } /* in principle, p1 and p2 are empty */ if (is_empty(p1) && is_empty(p2)) printf("What a joy!!"); else printf("Problem!?!?"); } </pre>
---	---

5. Application exercises

Exercise 1:

Enrich the abstract object STACK, concretized by a linked representation, by integrating the following operations:

- `nb_element`: returns the number of elements in the stack.
- `replace_top`: changes the top of the stack. It requires the stack to be non-empty.
- `clear`: clears all elements from the stack.

Exercise 2:

Write a C program that reads an expression with parentheses, assumed to be valid (number of opening parentheses = number of closing parentheses), and displays all the sub-expressions between parentheses starting from the innermost. For example, if the expression submitted to the program is:

$a + (b - (c * d) + 8.14) - (d * k)$ **The program should display:**

$(c * d)$

$(b - (c * d) + 8.14)$

$(d * k)$

Exercise 3:

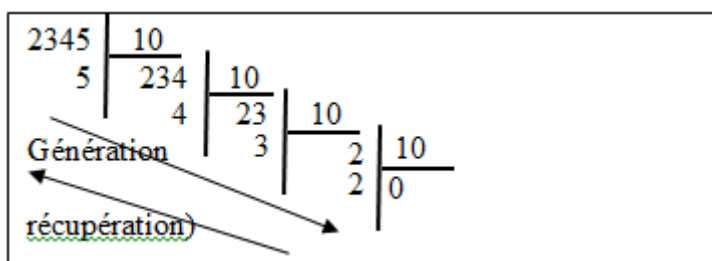
Write a C program that reads an integer and displays all the digits that compose it.

Example:

If the given number is 2345, then the desired program should display in order: 2, 3, 4, 5

Solution:

Idea: divide successively by integers



=> use the stack data structure

Correction Exercise 1:

The integration of new operations applicable to a data structure (SD) requires:

- Modification to the interface
- Modification to the implementation
- Application: abstract object framework
- Abstract object: pile.h, pile.c

Modification to pile.h:

```
/* Keep the fundamental operations unchanged: create_stack, is_empty, etc. */
```

```
unsigned nb_elements(void);
```

```
void clear(void);
```

```
void change_top(int);
```

Modification to pile.c:

```
unsigned nb_elemV1(void) {
```

```
    unsigned nb;
```

```
    nb = 0;
```

```
    while (!is_empty()) {
```

```
        nb++;
```

```
        pop();
```

```
    }
```

```
    return(nb);
```

```
}
```

```
static unsigned nb; /* number of elements in the stack */
```

```
unsigned nb_elements(void) {
```

```
    return nb;
```

```
}
```

```
void create_stack(void) {  
  
    ...  
  
    nb = 0;  
  
}  
  
void push(int info) {  
  
    ...  
  
    nb++;  
  
}  
  
void pop(void) {  
  
    ...  
  
    nb--;  
  
}  
  
void change_top(int new_value) {  
  
    assert(!is_empty());  
  
    top->key = new_value;  
  
}  
  
void clear(void) {  
  
    while (!is_empty()) {  
  
        pop();  
  
    }  
  
}
```

Note: The implementation of the `clear` operation is based on fundamental operations (here `is_empty` and `pop`). It is considered a "composed" operation, derived from the fundamental operations. On the other hand, `change_top` manipulates elements related to the physical representation of the stack. If the data structure stack is empty, then (implication) `nb_elements = 0`. Similarly, if the number of elements in the stack is zero, the stack is empty.

Correction Exercise 2:

Idea: A closing parenthesis is associated with the last opening parenthesis encountered, which implies LIFO => STACK.

```
#include<stdio.h>
#include<string.h>
#include<stdlib.h>
#include "stack.h" /* abstract object */
char *substring(char ch[], unsigned start, unsigned end) {
    /* returns the substring between ch[start] and ch[end] */
    unsigned i; /* between start and end */
    char *sub_str;
    char *p;
    sub_str = (char *) malloc(end - start + 2);
    p = sub_str;
    for (i = start; i <= end; i++) {
        *p = ch[i];
        p++;
    }
    *p = '\0';
    return(sub_str);
}
void main() {
    char ch[80]; /* input expression */
    char *sub_str; /* current expression within parentheses */
    unsigned i;
    printf("exp=");
    gets(ch);
    for (i = 0; !is_empty(); i++) {
        if (ch[i] == '(')
            push(i);
        else if (ch[i] == ')') {
            sub_str = substring(ch, last(), i);
            puts(sub_str);
            pop();
        }
    } /* end for */
} /* end main */
```

Correction Exercise 3:

```
#include <stdio.h>
#include "stack.h" /* abstract object */
void main(void) {
    unsigned nb;
    unsigned digit; /* 0...9 */
    printf("nb=");
    scanf("%u", &nb);
    create_stack();
    do {
        digit = nb % 10; /* remainder of nb divided by 10 */
        push(digit);
        nb = nb / 10;
```

```
    } while (nb != 0);

    /* usage */
    while (!is_empty()) {
        printf("%u\n", last());
        pop();
    }
} /* end main */
```